

Department of Computer Science and Engineering

PES UNIVERSITY

UE19CS202: Data Structures and its Applications (4-0-0-4-4)

Trees

Binary Trees: Basic Concept and Definitions

Dr. Shylaja S S
Ms. Kusuma K V

Binary Tree: is a Non Linear Data Structure

Definition: Finite set of elements that is either empty or is partitioned into three subsets

- First subset: is a single element, called the root
- Second subset: is a binary tree, called the left binary tree
- Third subset: is a binary tree, called the right binary tree

Figure 3 shows a Binary Tree with A as its root. Tree with nodes B, D, E, G form the left subtree and the tree with nodes C, F, H, I form the right subtree.

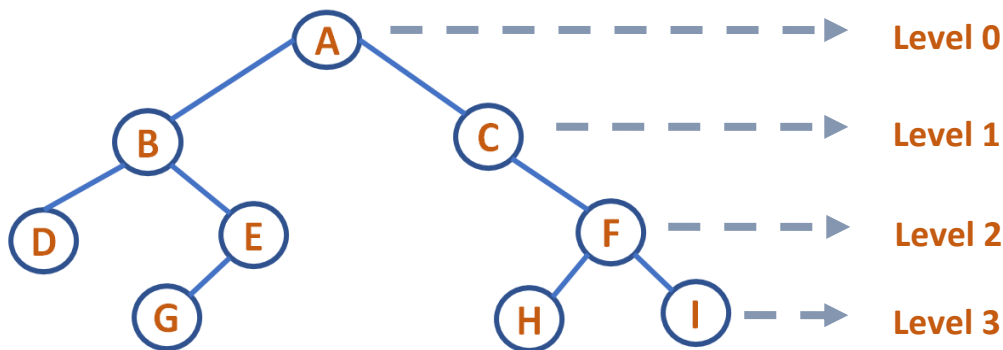


Figure 3: Binary Tree

Each element of a binary tree is called a **node** of the tree.

Left node B of A is called left child of A. Right node C of A is called the right child of A.

Two nodes are called **siblings** if they are left and right children of the same parent. A is called the parent of B and C. B and C are called siblings

A node which has no child is called **leaf node/external node**

A node which has a child is called the non **leaf node/internal node**

A node N1 is called the **ancestor** of a node N2 if N1 is either the parent of N2 or N1 is the parent of some ancestor of N2. A node N2 becomes the **descendent** of node N1. Descendent can be either the left descendent or the right descendent.

Level of a node: Root has Level 0; Level of any other node is one more than its parent.

Depth of a tree: Maximum level of any leaf in the tree (path length from the deepest leaf to the root)

Depth of a node: Path length from the node to the root

Height of a tree: Path length from the root node to the deepest leaf

Height of a node: Path length from the node to the deepest leaf

In Figure 3, D, G, H and I are leaf nodes. A, B, C, E and F are internal nodes. A is the ancestor of all the nodes in the tree. B is the left descendant of A. C is the right descendent of A.

Level of node A: 0, Level of node B, C: 1, Level of node D, E, F: 2, Level of node G, H, I: 3

Depth of tree: 3

Depth of node A: 0, Depth of node B, C: 1, Depth of node D, E, F: 2, Depth of node G, H, I: 3

Height of tree: 3

Height of Node A: 3, Height of Node B, C: 2, Height of Node E, F: 1, Height of Node D,G,H,I: 0

Figure 4 shows some structures that are not binary trees.

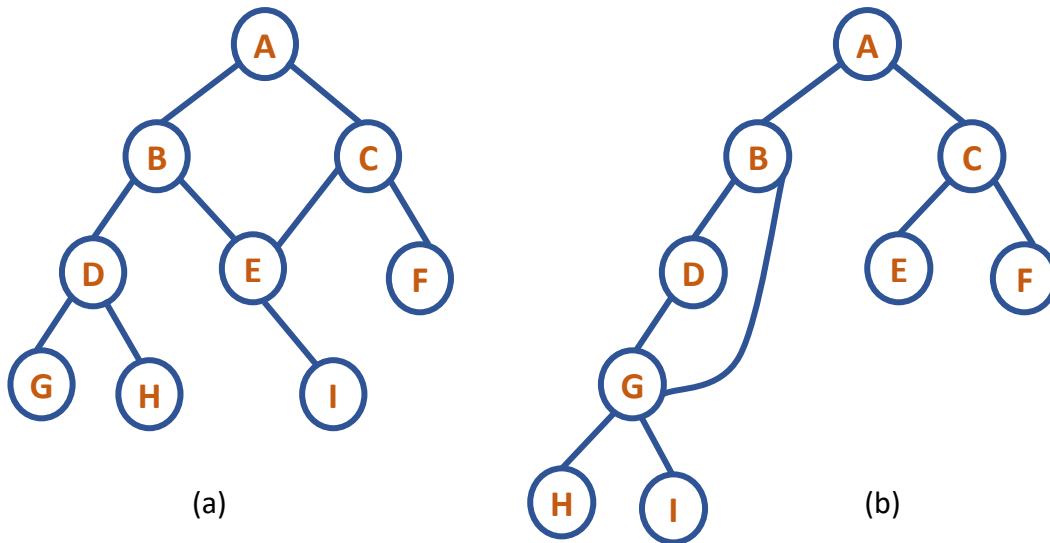


Figure 4: Structures that are not Binary Trees

Note: Different authors use the following tree terminologies in different ways.

Strictly Binary Tree: A Binary tree where every node has either zero/two children.

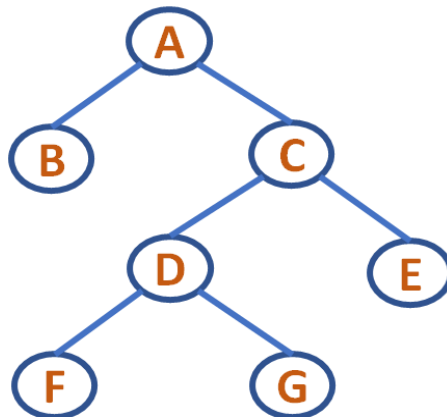


Figure 5: Strictly Binary Tree

Fully Binary Tree: A binary tree with all the leaves at the same level.

- If the binary tree has depth d , then there are 0 to d levels
- Total no. of nodes = $2^0 + 2^1 + \dots + 2^d = 2^{(d+1)} - 1$

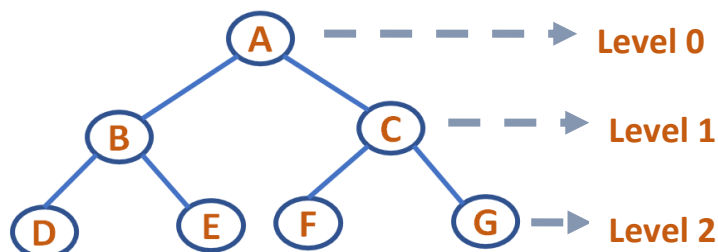


Figure 6: Full Binary Tree of depth 2

Complete Binary Tree: For a Complete Binary Tree with n nodes and depth d :

1. Any node n_d at level less than $d-1$ has two children
2. For any node n_d of the tree with a right descendent at level d , n_d must have a left child and every left descendent of n_d is either a leaf at level d or has two children

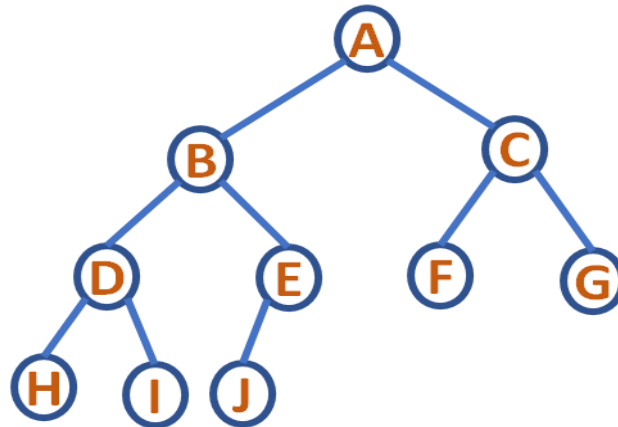


Figure 7: Complete Binary Tree

The binary tree of Figure 8(a) is not complete, since it contains leaves at levels 1, 2, and 3, thereby violating condition 1. The binary tree of Figure 8(b) satisfies condition 1, since every leaf is either at level 2 or at level 3. However, condition 2 is violated, since A has a right descendent at level 3 (J) but also has a left descendant that is a leaf at level 2 (E).

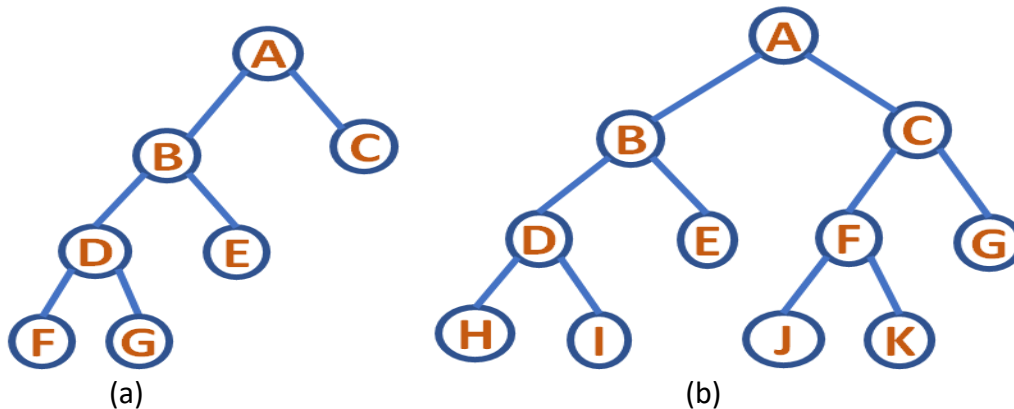


Figure 8: Structures that are not Complete Binary Trees

Binary Tree Properties

- Every node except the root has exactly one parent
- A tree with n nodes has $n-1$ edges (every node except the root has an edge to its parent)
- A tree consisting of only root node has height of zero
- The total number of nodes in a full binary tree of depth d is $2^{(d+1)} - 1$, $d \geq 0$
- For any non-empty binary tree, if n_0 is the number of leaf nodes and n_2 the nodes of degree 2, then $n_0 = n_2 + 1$

Department of Computer Science and Engineering

PES UNIVERSITY

UE19CS202: Data Structures and its Applications (4-0-0-4-4)

Trees
Binary Trees: Traversal

Dr. Shylaja S S
Ms. Kusuma K V

Binary Tree Traversal:

Traversal is the process of moving through all the nodes in a binary tree and visiting each one in turn. The action taken when each node is visited depends on the application; here we shall print the content of each node on visiting.

Tree being a non linear data structure, there are many different ways in which we could traverse all the nodes. When we write an algorithm to traverse a binary tree, we shall almost always wish to proceed so that the same rules are applied at each node, and we thereby adhere to a general pattern.

At a given node, then, there are three tasks we shall wish to do in some order. They are:

- 1) Visiting a node, let us denote it by V
- 2) Traversing the left subtree, let us denote it by L
- 3) Traversing the right subtree, let us denote it by R

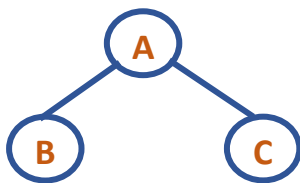
The key distinction in traversal orders is to decide if we are to visit the node itself before traversing either subtree, between the subtrees, or after traversing both the subtrees.

The three tasks can be done in six different ways: VLR, LVR, LRV, VRL, RVL, RLV.

By Standard convention, these 6 ways are reduced to three by permitting only the ways in which the left subtree is traversed before the right. The three mirror images are clearly similar. The three ways with left subtree before right subtree are given the following names: VLR – Preorder, LVR – Inorder, LRV – Postorder

These three names are chosen according to the step at which the given node is visited. With preorder traversal, the node is visited before the subtrees, with inorder traversal, node is visited between the left and right subtree, and with postorder traversal, the node is visited after both the subtrees.

Eg1: Consider a simple example of a Binary Tree shown below:



Preorder Traversal: A B C

Inorder Traversal: B A C

Postorder Traversal: B C A

Binary Tree Recursive Traversal

Preorder Traversal:

- Root Node is visited before the subtrees
- Left subtree is traversed in preorder
- Right subtree is traversed in preorder

```
void preorder(NODE* root)
{
    if(root!=NULL)
    {
        printf("%d ",root->info);
        preorder(root->left);
        preorder(root->right);
    }
}
```

Inorder Traversal:

- Left subtree is traversed in Inorder
- Root Node is visited
- Right subtree is traversed in Inorder

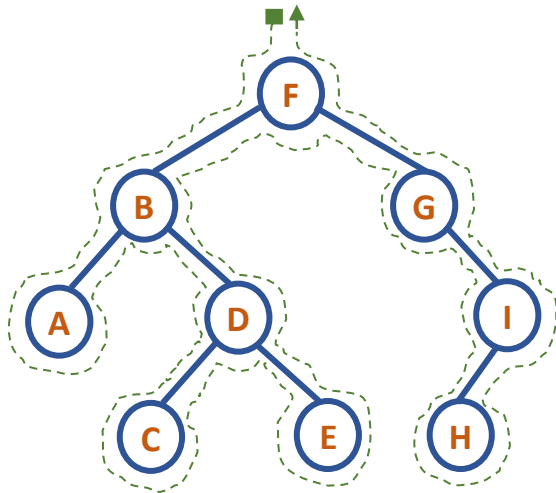
```
void inorder(NODE* root)
{
    if(root!=NULL)
    {
        inorder(root->left);
        printf("%d ",root->info);
        inorder(root->right);
    }
}
```

Postorder Traversal:

- Left subtree is traversed in postorder
- Right subtree is traversed in postorder
- Root Node is visited

```
void postorder(NODE* root)
{
    if(root!=NULL)
    {
        postorder(root->left);
        postorder(root->right);
        printf("%d ",root->info);
    }
}
```

Eg2: Write the Preorder, Inorder and Postorder Traversal for the Binary Tree given below:



Preorder Traversal: F B A D C E G I H
 Inorder Traversal: A B C D E F G H I
 Postorder Traversal: A C E D B H I G F

Binary Tree Iterative Traversal

1) Inorder Traversal

//Iterative Inorder Traversal

```

iterativeInorder(root)
s = emptyStack
current = root
do {
    while(current != null)
    {
        /* Travel down left branches as far as possible saving pointers to
        nodes passed in the stack*/
        push(s, current)
        current = current->left
    } //At this point, the left subtree is empty
    poppedNode = pop(s)
    print poppedNode ->info           //visit the node
    current = poppedNode ->right     //traverse right subtree
} while(!isEmpty(s) or current != null)
  
```

2) Preorder Traversal

//Iterative Preorder Traversal

```

iterativePreorder(root)
current=root
if (current == null)
    return
s = emptyStack
push(s, current)
  
```

```
while(!isEmpty(s)) {
    current = pop(s)
    print current->info
    //right child is pushed first so that left is processed first
    if(current->right !=NULL)
        push(s, current->right)
    if(current->left !=NULL)
        push(s, current->left)
}
```

3) Postorder Traversal

//Iterative Postorder Traversal

```
iterativePostorder(root)
s1 = emptyStack
s2 = emptyStack
push(s1, root)

while(!isEmpty(s1)) {
    current = pop(s1)
    push(s2,current)
    if(current->left !=NULL)
        push(s1, current->left)
    if(current->right !=NULL)
        push(s1, current->right)
}
while(!isEmpty(s2)) {      //Print all the elements of stack2
    current = pop(s2)
    print current->info
}
```

Department of Computer Science and Engineering

PES UNIVERSITY

UE19CS202: Data Structures and its Applications (4-0-0-4-4)

Trees
N-ary Trees and Forest

Dr. Shylaja S S
Ms. Kusuma K V

Tree: is a Non Linear Data Structure

Definition: Finite nonempty set of elements

- One element is the root
- Remaining elements are partitioned into $m \geq 0$ disjoint subsets each of which is itself a tree

Ordered Tree: a tree in which subtrees of each node form an ordered set

- In such a tree we define first, second, ..., last child of a particular node
- First child is called the oldest child and last child the youngest child

Figure 1 shows an ordered tree with A as its root, B is the oldest child and D is the youngest child of A. E is the oldest and F is the youngest child of B.

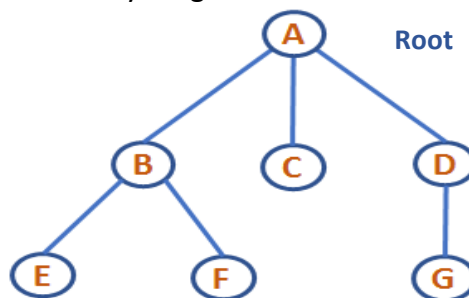


Figure 1: Ordered Tree

Note: Here on we will refer an ordered tree as just a tree

n-ary tree: A rooted tree in which each node has no more than n children.

A binary tree is an n-ary tree with $n=2$. Figure 2 shows an n-ary tree with $n=5$.

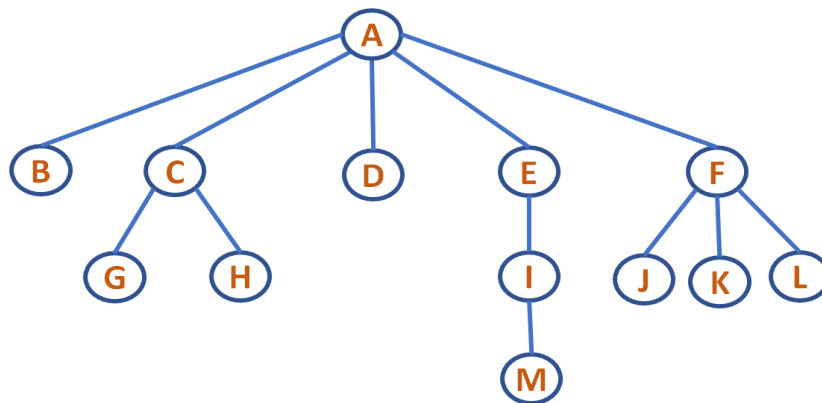


Figure 2: n-ary tree, $n=5$

Forest: is an ordered set of ordered trees

In the representation of a binary tree, each node contains an information field and two pointers to its two children. Trees may be represented as an array of tree nodes or a dynamic variable may be allocated for each node created. But how many pointers should a tree node contain? The number of children a node can have varies and may be as large or as small as desired.

```
#define MAXCHILD 20
struct treenode{
    int info;
    struct treenode *child[MAX];
};
where MAX is a constant
```

Restriction with the above implementation is that a node cannot have more than MAX children. Therefore the tree cannot be expanded.

Consider an alternative implementation as follows: All the children of a given node are linked and only the oldest child is linked to the parent

A node has link to first child and a link to immediate sibling

```
struct treenode{
    int info;
    struct treenode *child;
    struct treenode *sibling;
};
```

Conversion of an n-ary tree to a Binary Tree

Using Left Child – Right Sibling Representation an n-ary tree can be converted to a binary tree as follows:

1) Link all the siblings of a node

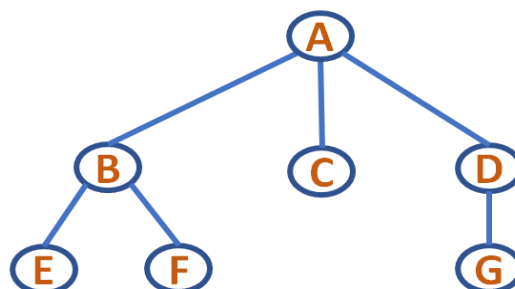
2) Delete all links from a node to its children except for the link to its leftmost child

The **left child in binary tree** is the node which is the oldest child of the given node in an n-ary tree, and the **right child is the node to the immediate right of the given node** on the same horizontal line. Such a binary tree will not have a right sub tree.

The node structure looks as shown below:

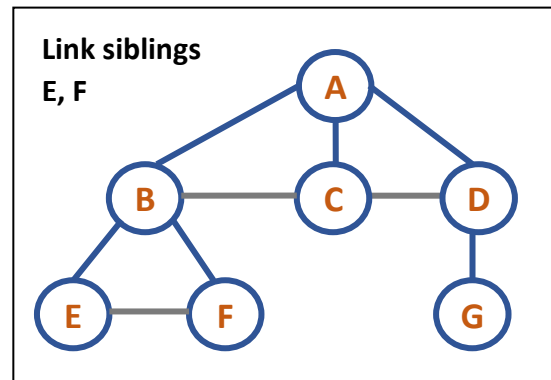
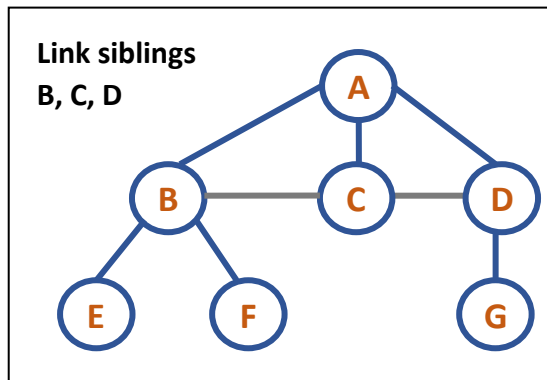
Data	
Left Child	Right Sibling

Consider the 3-ary tree shown below:

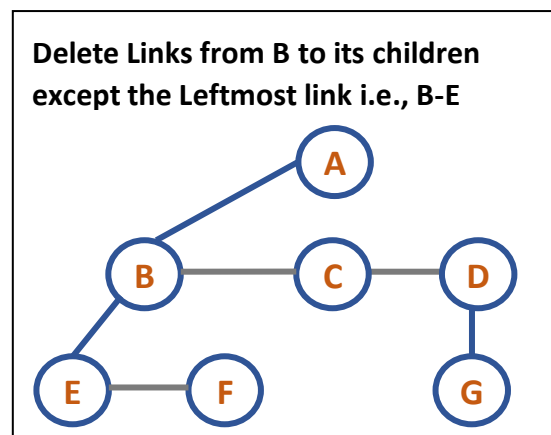
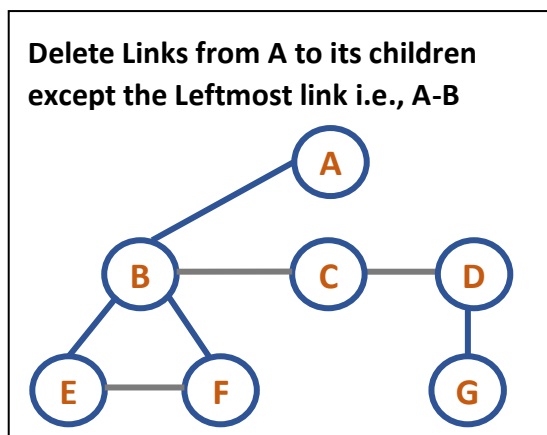


3 - ary tree

First step in the conversion is to Link all the siblings of a node.



Second step is to delete all the links from a node to its children except for the link to its leftmost child.



There are no more multiple links from any parent node to its children. The tree so obtained is the binary tree. But it doesn't look like one. Use the left child-right sibling relationship and make the tree look like a binary tree i.e., for any given node, its leftmost child will become the left child and immediate sibling becomes the right child. The binary tree so obtained will always have an empty right subtree for the root node. This is because the root of the tree we are transforming has no siblings.

For node A: Left child is B and has no Right child

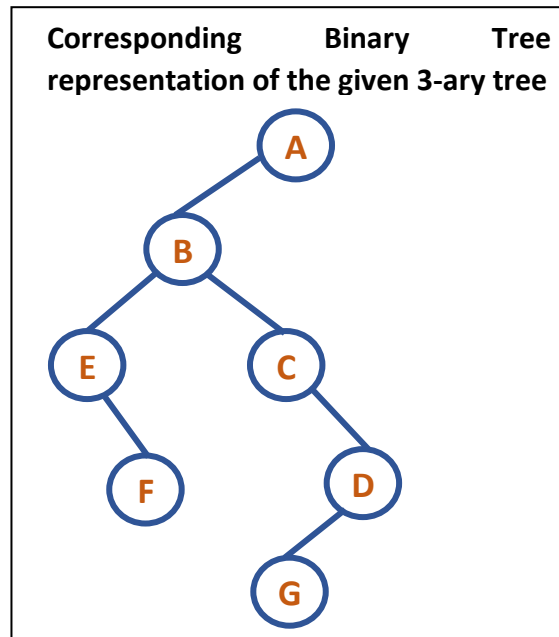
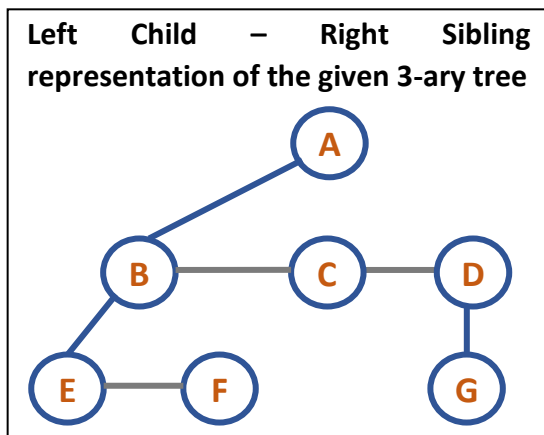
For Node B: Left child is E and Right child is C

For Node C: No Left child and Right child is D

For node D: Left child is G and has no Right child

For Node E: No Left child and Right child is F

For Node F and Node G: No children (No Left child: because they do not have a child and no Right child: because they do not have a sibling towards right)



Conversion of a Forest to a Binary Tree

In the n-ary tree to binary tree conversion as stated above we saw that the right subtree for the root node of the binary tree is always empty. This is because the root of the tree we are transforming has no siblings.

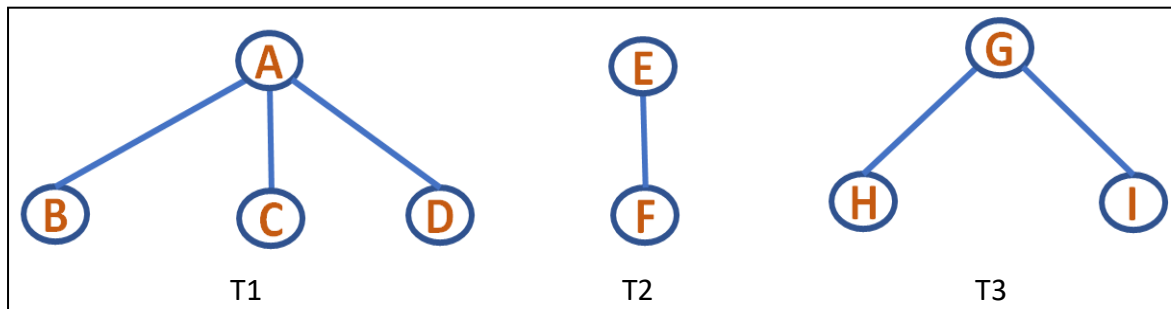
On the other hand, if we have a forest then these can all be transformed into a single binary tree as follows:

- 1) First obtain the binary tree representation of each of the trees in the forest
- 2) Link all the binary trees together through the right sibling field of the root nodes

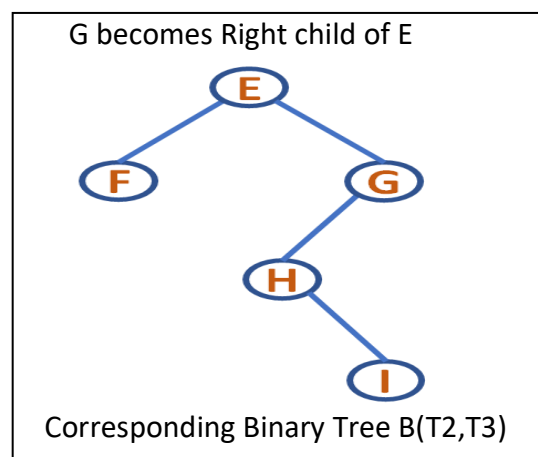
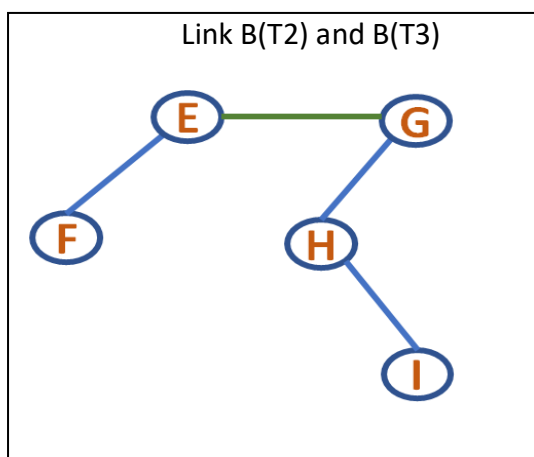
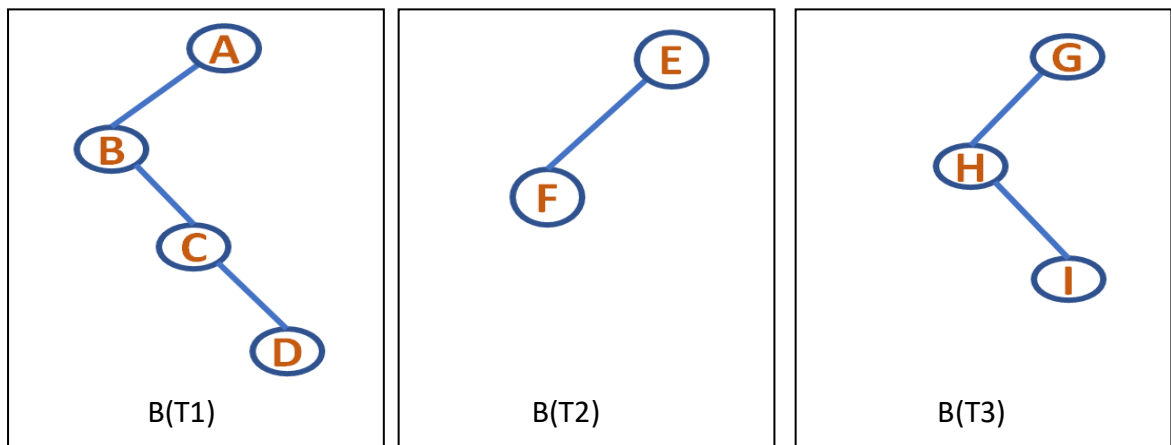
Conversion of a Forest to a Binary Tree can be formally defined as follows:

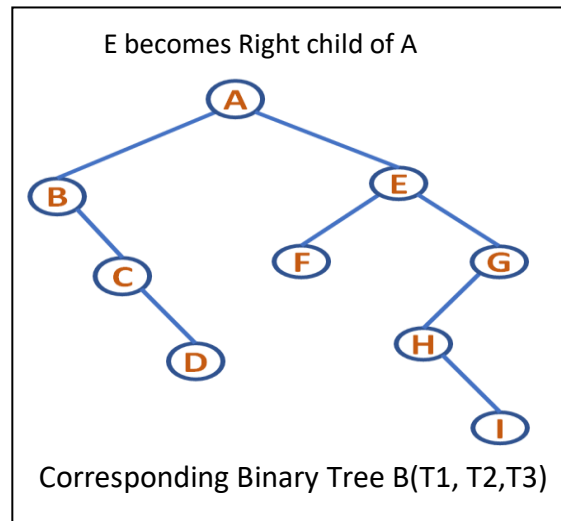
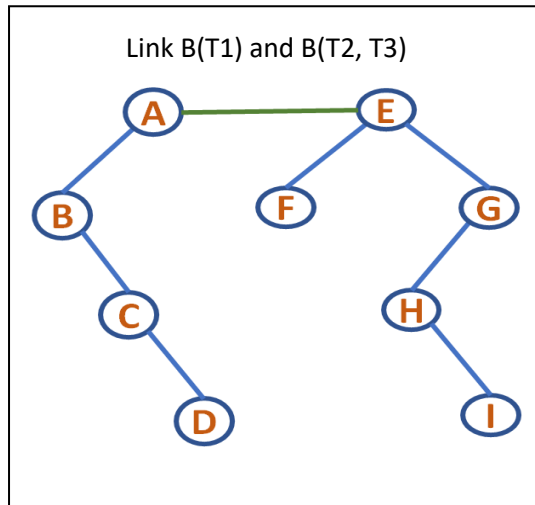
- If $T_1, \dots, T_n$ is a forest of n trees, then the binary tree corresponding to this forest, denoted by $B(T_1, \dots, T_n)$:
 - is empty if $n = 0$
 - has root equal to root (T_1)
 - has left subtree equal to $B(T_{11}, T_{12}, \dots, T_{1m})$ where $T_{11}, \dots, T_{1m}$ are the subtrees of root (T_1)
 - has right subtree $B(T_2, \dots, T_n)$

Consider the following Forest with three Trees:



Corresponding Binary Trees:





Tree Traversal

The traversal methods for binary trees induce traversal methods for forests. The preorder, inorder, or postorder traversals of a forest may be defined as the preorder, inorder, or postorder traversals of its corresponding binary tree.

```
struct treenode{
    int info;
    struct treenode *child;
    struct treenode *sibling;
};
```

With the treenode implemented as having pointers to first child and immediate sibling, the traversal preorder, inorder and postorder for a tree are defined as below:

Preorder:

1. Visit the root of the first tree in the forest
2. Traverse in preorder the forest formed by the subtrees of the first tree, if any
3. Traverse in preorder the forest formed by the remaining trees in the forest, if any

```
void preorder(TREE *root)
{
    if(root!=NULL)
    {
        printf(" %d ",root->info);
        preorder(root->child);
        preorder(root->sibling);
    }
}
```


Inorder:

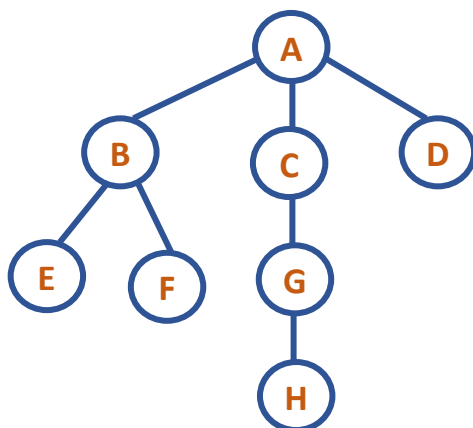
1. Traverse in inorder the forest formed by the subtrees of the first tree, if any
2. Visit the root of the first tree in the forest
3. Traverse in inorder the forest formed by the remaining trees in the forest, if any

```
void inorder(TREE *root)
{
    if(root!=NULL)
    {
        inorder(root->child);
        printf(" %d ",root->info);
        inorder(root->sibling);
    }
}
```

Postorder:

1. Traverse in postorder the forest formed by the subtrees of the first tree, if any
2. Traverse in postorder the forest formed by the remaining trees in the forest, if any
3. Visit the root of the first tree in the forest

```
void postorder(TREE *root)
{
    if(root!=NULL)
    {
        postorder(root->child);
        postorder(root->sibling);
        printf(" %d ", root->info);
    }
}
```



Traversal of the above n-ary Tree:

Preorder: ABEFCGHD

Inorder: EFBHGCDA

Postorder: FEHGCDBA